

Než začneš porovnávat algoritmy, zopakuj si mocniny dvojky a půlení, rozdíl mezi lineárním a kvadratickým růstem, bity a v Pythonu cyklus se seznamem a funkce s testy. Právě z nich se efektivita počítá.

1. Mocniny dvojky a půlení ZŠ

Kde to najdeš: Začátečníci H12 Mocniny

Binární vyhledávání v každém kroku zmenší úsek na polovinu. Kolik kroků potřebuje, zjistíš z mocnin dvojky:

$2^{10} = 1\,024$, takže tisíc prvků lze půlit asi desetkrát, než zbude jediný. Mocniny dvojky je dobré znát z paměti aspoň do 2^{10} . Platí $2^{20} \approx$ milion a $2^{30} \approx$ miliarda.

$$2^{10} = 1\,024 \approx 10^3$$

Příklad: Kolikrát musíš rozpůlit seznam 1 000 prvků, než zbude jeden?

1. $2^9 = 512 < 1\,000$, ale $2^{10} = 1\,024 \geq 1\,000$.
2. Stačí 10 půlení.

Zkus sám:

1. Vypočítej 2^5 a 2^8 .
2. Kolikrát musíš rozpůlit 64 prvků, než zbude jeden?
3. Kolik je přibližně 2^{20} ?

2. Lineární a kvadratický růst ZŠ

Kde to najdeš: Začátečníci H9 Přímá a nepřímá úměrnost, Začátečníci H12 Mocniny

U lineárního algoritmu (projde seznam jednou) roste počet kroků přímo úměrně velikosti vstupu: dvakrát větší vstup = dvakrát víc kroků. U kvadratického algoritmu (každý prvek se porovnává s každým) roste počet kroků s druhou mocninou: dvakrát větší vstup = čtyřikrát víc kroků, desetkrát větší = stokrát víc. Proto se rozdíl ukáže až pro velké vstupy.

$$t \propto n \text{ lineární růst} \quad t \propto n^2 \text{ kvadratický růst}$$

Příklad: Řazení výběrem trvalo pro 1 000 prvků 0,05 s. Kolik zhruba potrvá pro 2 000 a pro 10 000 prvků?

1. Algoritmus je kvadratický: $2\times$ víc prvků $\rightarrow 2^2 = 4\times$ déle, tedy asi 0,2 s.
2. $10\times$ víc prvků $\rightarrow 10^2 = 100\times$ déle, tedy asi 5 s.

Zkus sám:

1. Vstup se zvětší třikrát. Kolikrát víc kroků udělá kvadratický algoritmus?
2. Lineární algoritmus běží pro 10^5 prvků 0,3 s. Kolik poběží pro 10^6 prvků?
3. Kvadratický algoritmus běží pro 10^4 prvků 1 s. Kolik poběží pro 10^5 prvků?

3. Bity a otázky ano/ne SŠ

Kde to najdeš: 1.+2. H34 Bity a informace

Odpověď na otázku ano/ne nese 1 bit informace a dobře položená otázka vyřadí polovinu možností. Mezi N možnostmi proto najdeš tu správnou nejvýš na k otázek, kde k je nejmenší číslo s $2^k \geq N$. Totéž platí pro binární vyhledávání: jedno porovnání je jedna otázka „je hledané číslo menší?“.

$$2^k \geq N \text{ počet otázek } k \text{ pro } N \text{ možností}$$

Příklad: Kolik otázek ano/ne stačí na uhodnutí čísla od 1 do 1 000 000?

1. $2^{19} = 524\,288 < 1\,000\,000$.
2. $2^{20} = 1\,048\,576 \geq 1\,000\,000 \rightarrow$ stačí 20 otázek.

Zkus sám:

1. Kolik otázek stačí na číslo od 1 do 100?
2. Kolik otázek stačí na číslo od 1 do 16?

4. Cyklus, seznam a lineární vyhledávání SS

Kde to najdeš: 1.+2. H49 Cykly a seznamy

Lineární vyhledávání prochází seznam od začátku a každý prvek porovná s hledanou hodnotou; když ji najde, vrátí její index, jinak po projití celého seznamu vrátí -1. V nejhorším případě udělá tolik porovnání, kolik má seznam prvků, v průměru asi polovinu. Funguje i na neseřazeném seznamu.

Příklad: Kolik porovnání udělá lineární vyhledávání čísla 16 v seznamu [4, 8, 15, 16, 23] a co vrátí?

1. Porovná postupně 4, 8, 15, 16 – shoda nastane ve 4. porovnání.
2. Vrátí index 3 (indexy začínají nulou).

Zkus sám:

1. Kolik porovnání udělá lineární vyhledávání v nejhorším případě v seznamu 1 000 prvků?
2. Kolik porovnání udělá v průměru, když hledaný prvek v seznamu 1 000 prvků určitě je?

5. Funkce s parametry a testy assert SS

Kde to najdeš: 1.+2. H52 Podprogramy, 1.+2. H47 Testování a ladění

Algoritmus zapíšeš jako funkci s parametry, takže ho můžeš použít na různé vstupy – to je zobecnění. Správnost ověříš testy: příkaz assert podmínka neudělá nic, když podmínka platí, a ohlásí chybu AssertionError, když neplatí. Po každém vylepšení algoritmu spust' stejné testy znovu.

Příklad: Napiš test, který ověří, že tvoje funkce serad(s) řadí stejně jako vestavěné sorted.

1. Připrav několik vstupů: prázdný seznam, jeden prvek, opakující se čísla, zamíchaný seznam.
2. Pro každý vstup s napiš assert serad(s) == sorted(s).
3. Když se nic nevypíše, testy prošly.

Zkus sám:

1. Co se stane, když neplatí podmínka v příkazu assert?
2. Co udělá assert sorted([3, 1, 2]) == [1, 2, 3]?

Kde se to doučit

- **Algoritmy.net – Binární vyhledávání** – Princip půlení intervalu a počet kroků.
<https://www.algoritmy.net/article/21/Binarni-vyhledavani>
- **VisuAlgo – řadicí algoritmy** – Animace bublinkového řazení a řazení výběrem.
<https://visualgo.net/en/sorting>
- **Algoritmy.net – Selection sort (řazení výběrem)** – Postup řazení výběrem krok za krokem.
<https://www.algoritmy.net/article/4/Selection-sort>

Výsledky ke kontrole

- **1. Mocniny dvojky a půlení:** 1) 32 a 256; 2) 6krát; 3) asi milion (1 048 576)
- **2. Lineární a kvadratický růst:** 1) 9krát; 2) asi 3 s; 3) asi 100 s
- **3. Bity a otázky ano/ne:** 1) 7; 2) 4
- **4. Cyklus, seznam a lineární vyhledávání:** 1) 1 000; 2) asi 500
- **5. Funkce s parametry a testy assert:** 1) program skončí chybou AssertionError; 2) nic – podmínka platí, test prošel